

Справочник по системе команд учебного компьютера для лабораторных работ (подробный)

Сайт: Образовательный портал "Электронный университет ВГУ"

Курс: Архитектура ЭВМ (Толстобров А. П., Ветохин В. В.)

Книга: Справочник по системе команд учебного компьютера для лабораторных работ (подробный)

Оглавление

- [Введение](#)
 - [1. Модель PDP11](#)
 - [1.1 Регистры общего назначения](#)
 - [1.2 Регистр состояния процессора \(РСП\)](#)
 - [1.3. Обращение к памяти и распределение адресов канала](#)
 - [1.4 Обмен данными между внешними устройствами и ЭВМ](#)
 - [2. Система команд учебной ЭВМ и методы адресации](#)
 - [2.1 Общие понятия](#)
 - [2.2. Формат одноадресных команд](#)
 - [2.3. Формат двухадресных команд](#)
 - [2.4. Методы прямой адресации](#)
 - [2.5. Методы косвенной адресации](#)
 - [2.6. Использование счетчика команд \(РС\) в качестве РОН](#)
 - [3. Обозначения, используемые при описании команд](#)
 - [4. Выполнение байтовых команд](#)
 - [5. Одноадресные команды](#)
 - [6. Двухадресные команды](#)
 - [7. Команды управления программой](#)
 - [7.1. Команды ветвления](#)
 - [7.2. Команды обращения к подпрограмме и выхода из подпрограммы](#)
 - [7.3. Команды прерывания](#)
 - [7.4. Команды управления процессором](#)
-

Введение

Целью проведения практических занятий по курсу «Архитектура ЭВМ» является изучение основ организации и архитектуры ЭВМ на примере программной реализации учебной ЭВМ на

базе процессорного семейства, ставшего для многих учебных изданий классическим, PDP-11.

Во время практических занятий студент использует для выполнения большинства заданий программную модель PDP11 в среде операционной системы Windows.

1. Модель PDP11

Модель PDP11, написанная на языке Си (MS Visual C++), выполняется в среде ОС Windows и имеет следующие технические данные:

- Система счисления для чисел и команд - двоичная.
- Разрядность для чисел и команд - 16 двоичных разрядов.
- Объем адресуемой оперативной памяти - 32К 16-разрядных слов.
- Число регистров общего назначения - 8.
- Система команд: безадресная, одноадресная, двухадресная.
- Виды адресации: регистровая, косвенно-регистровая, автоинкрементная, косвенно-автоинкрементная, автодекрементная, косвенно-автодекрементная, индексная и косвенно-индексная.
- Обработка внешних и внутренних прерываний выполняется с помощью памяти магазинного типа (стека).

Структурная схема учебной ЭВМ представлена на Рис. 1.1.

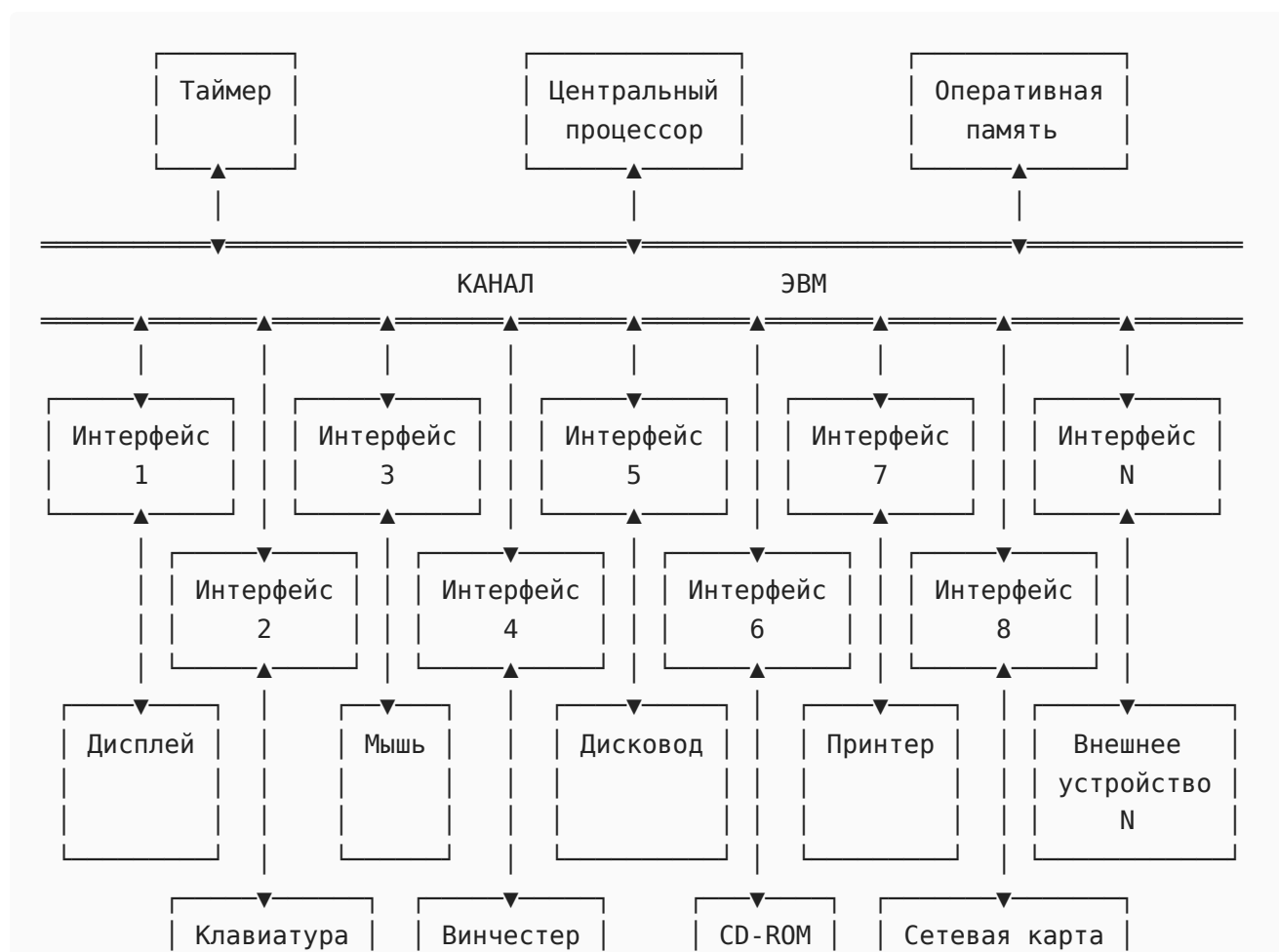


Рис. 1.1 Структурная схема учебной ЭВМ

1.1 Регистры общего назначения

Модуль центрального процессора учебной ЭВМ содержит 16-разрядные регистры общего назначения (РОН), используемые для выборки операндов и записи результатов при выполнении и арифметико-логических операций аналогично ячейкам памяти и регистрам внешних устройств.

Два из восьми имеющихся регистров общего назначения R0 - R7 имеют, кроме того, специальное назначение. Регистр R6 - *Указатель Стека (УС) (Stack Pointer - SP)* содержит адрес последней заполненной ячейки стека.

Регистр R7 служит *Счетчиком Команд (СК) (Program Counter - PC)* и содержит адрес ячейки памяти, из которой процессор выбирает очередную команду для выполнения. В связи с этим обычно этот регистр (R7) используется только для адресации и не используется как накопительный регистр для хранения операндов. Процессор работает таким образом, что после выборки из памяти ЭВМ команды по указанному в R7 (PC) адресу содержимое этого регистра, т.е. адрес очередной команды, автоматически увеличивается на два. Благодаря этому после выполнения очередной команды в регистре R7 будет находиться адрес следующей по порядку команды.

1.2 Регистр состояния процессора (РСП)

Регистр состояния процессора содержит информацию о текущем состоянии процессора. Это информация (слово состояния процессора - ССП) о значении разрядов (кодов) условий ветвления программы, зависящих от результата выполнения команды, текущем приоритете процессора и др.



Рис. 1.2 Формат слова состояния процессора

1.3. Обращение к памяти и распределение адресов канала

Весь обмен информацией и управляющими сигналами между различными устройствами и ЭВМ осуществляется через единый канал передачи информации. 16-разрядный код адреса позволяет обращаться к 32К 16-разрядных ячеек. Старшие 4К адресов (28К-32К) отведены под регистры внешних устройств. Распределение адресов канала показано на Рис. 1.3. Все адре

са даны в восьмеричном коде. Буква "К" используется для обозначения числа, равного $2^{10} = 1024$.

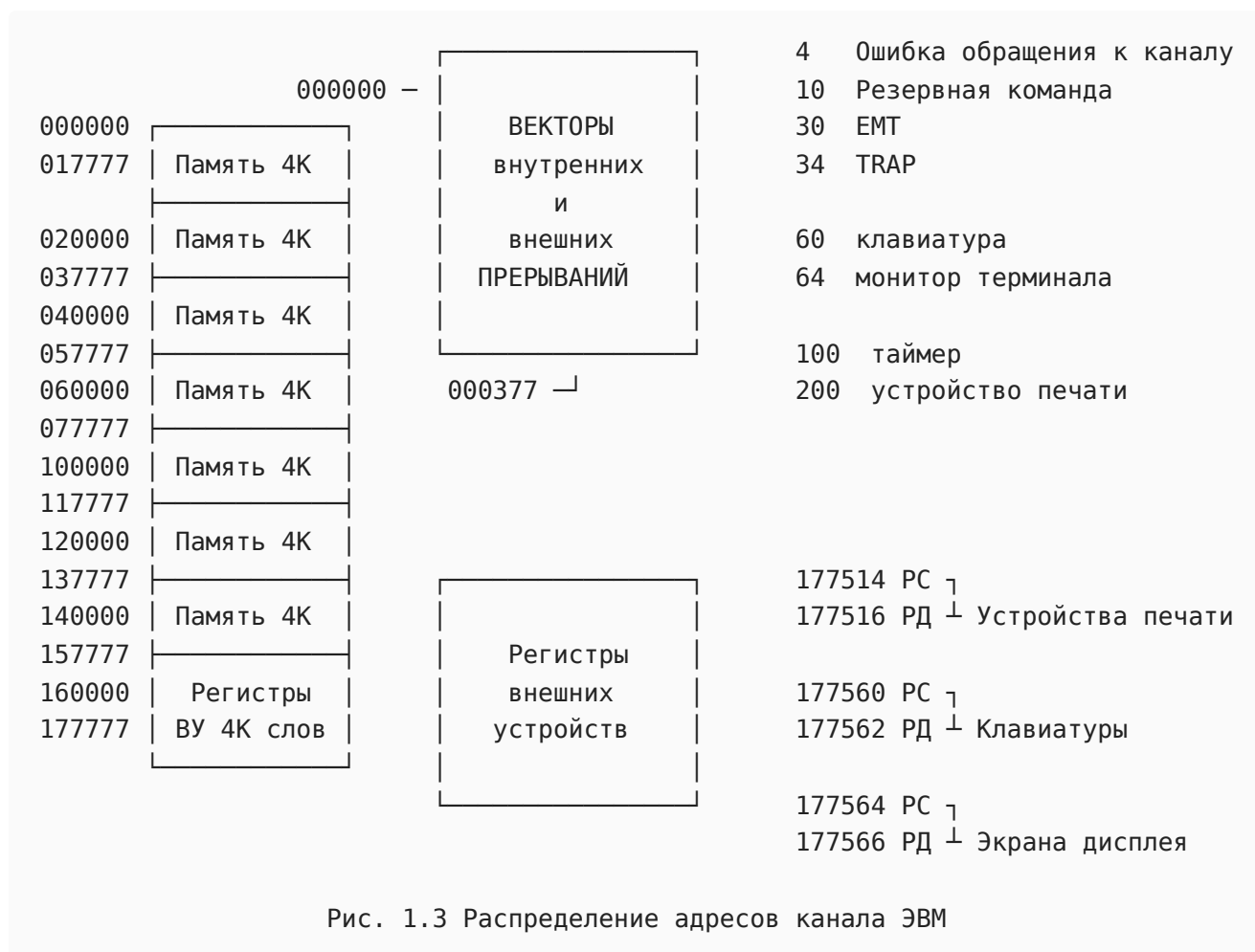
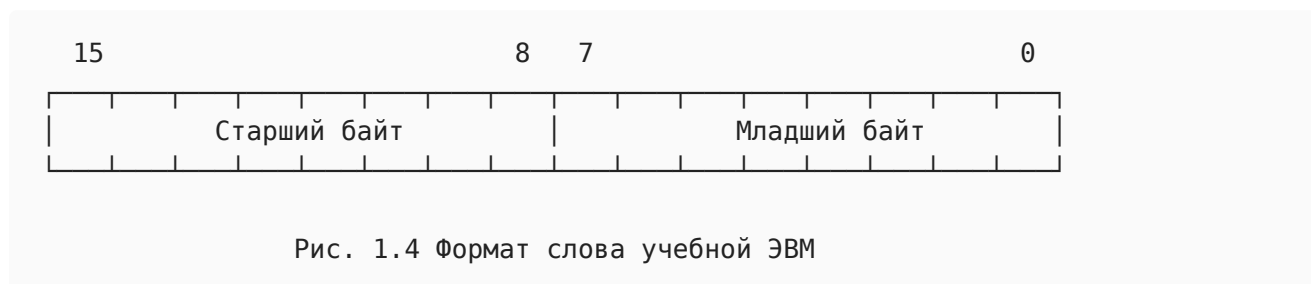


Рис. 1.3 Распределение адресов канала ЭВМ

Канал ЭВМ позволяет адресоваться к 32К 16-разрядных слов или к 64К байт. Ячейки памяти с 000000 по 000376 зарезервированы под векторы прерывания и использовать их для других целей не рекомендуется. Для каждого вектора необходимы две 16-разрядные ячейки, поэтому адреса векторов прерываний являются четными и заканчиваются на 0 или на 4. Последние 4К 16-разрядных адресов обычно отводятся для регистров внешних устройств, поэтому максимальный объем реальной памяти равен 28К 16-разрядных слов. Однако пользователь не обязан использовать все адреса этого пространства для этой цели и может руководствоваться соображениями необходимости.

Как показано на Рис. 1.4, машинное 16-разрядное слово делится на старший и младший байты. Ячейки, содержащие полные слова, всегда имеют четные адреса. Младшие байты слов хранятся в ячейках с четными адресами, а старшие с нечетными (Рис. 1.4).



СЛОВО		адреса	БАЙТ	адреса
байт	байт			
Старший	Младший	000000	Младший	000000
Старший	Младший	000002	Старший	000001
Старший	Младший	000004	Младший	000002
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
Старший	Младший	017770	Младший	017774
Старший	Младший	017772	Старший	017775
Старший	Младший	017774	Младший	017776
Старший	Младший	017776	Старший	017777

Рис. 1.5 Организация памяти ЭВМ по словам и по байтам для первых 4К адресов

1.4 Обмен данными между внешними устройствами и ЭВМ

Канал ЭВМ обеспечивает три типа обмена данными - это программный обмен, обмен в режиме прямого доступа к памяти и обмен в режиме прерывания программы. Обмен информацией между центральным процессором и внешними устройствами выполняется при помощи стандартных циклов обращения к каналу. Для организации обмена каждое внешнее устройство должно иметь один или несколько регистров (регистры данных, регистры состояния и др.), адреса которых определяет пользователь.

Как правило, регистры внешних устройств имеют четные адреса, однако при помощи байтовых команд можно обращаться к любому байту 16-разрядного регистра. Каждое внешнее устройство может иметь несколько различных регистров.

Регистр состояния (РС) содержит информацию об операции, выполняемой внешним устройством, характеризует состояние внешнего устройства и участвует в операциях по предоставлению прерывания.

Регистр данных (РД) используется при обмене данными между центральным процессором и внешним устройством.

Различные разряды регистров внешних устройств могут выполнять различные функции. Некоторые из них могут использоваться как для записи, так и для считывания информации, другие - только для записи или только для считывания. Типичным примером разряда, используемого для считывания и для записи, является разряд разрешения прерывания в регистре состояния внешнего устройства. Примером разряда, только принимающего информацию, является разряд пуска, а разрядом, используемым только для считывания, - разряд ошибки регистра состояния внешнего устройства. Регистры данных внешних устройств, как правило, являются обычными накопительными регистрами, и их формат определяется только требованиями пользователя. Формат регистров состояния внешних устройств показан на Рис. 1.6. Данный формат не является обязательным, но желательным для обеспечения унификации операций, выполняемых при обращении к внешним устройствам. Заметим, что регистры состояния большинства внешних устройств имеют меньше 16 разрядов.

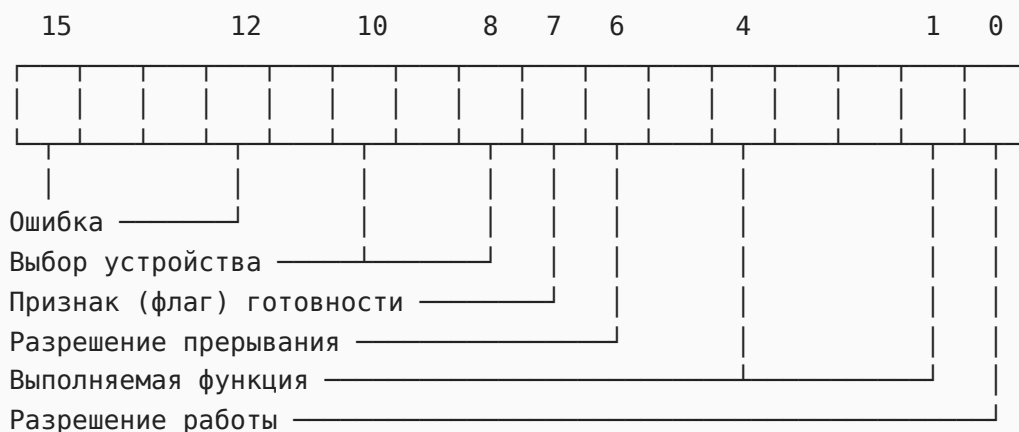


Рис. 1.6 Формат регистра состояния внешнего устройства

2. Система команд учебной ЭВМ и методы адресации

2.1 Общие понятия

Команды ЭВМ, предназначенные для обработки данных, помимо кода выполняемой операции и должны тем или иным образом указывать местонахождение (адрес) этих данных (операндов) в памяти ЭВМ. В связи с этим большое значение имеют реализованные в конкретной ЭВМ методы адресации операндов, т.е. способы указания в машинной команде местонахождения операндов в памяти ЭВМ.

Способы адресации можно классифицировать на прямые и косвенные. При прямом способе адресации исполнительный адрес берется непосредственно из команды или вычисляется с использованием значения указанного в команде и содержимого какого-либо регистра.

Косвенный способ адресации предполагает, что в команде содержится значение косвенного адреса, т.е. адреса ячейки памяти, в которой находится окончательный исполнительный адрес.

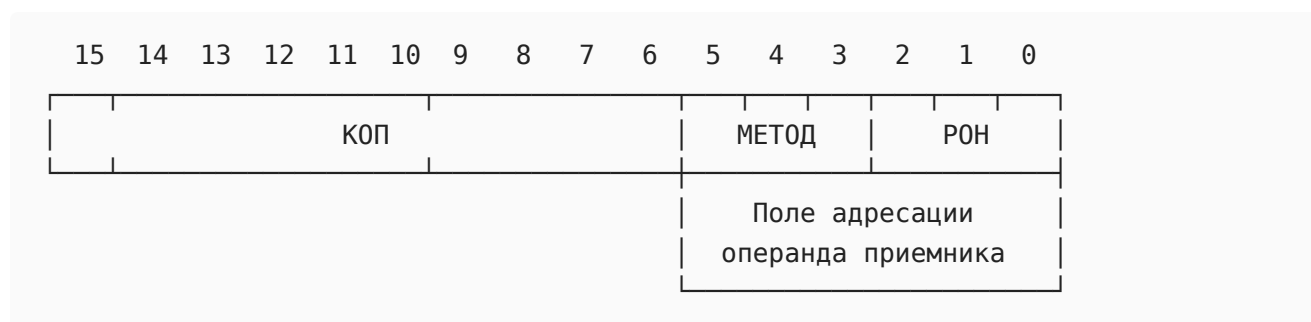
При реализации методов адресации ЭВМ существенным образом используются регистры центрального процессора (РОН). Далее будем использовать термин *адресный регистр* для обозначения любого регистра центрального процессора, содержащего адрес.

В командах обработки данных может быть указано местонахождение от одного до нескольких операндов, используемых при выполнении конкретной операции. В учебной ЭВМ используются одноадресные и двухадресные команды. При этом обычно различают операнд-источник и операнд-приемник. Операнд-источник это содержимое ячейки памяти или регистра, которое используется при выполнении указанной в команде операции и которое в процессе выполнения команды не изменяется. Операнд-приемник это ячейка памяти или РОН, содержимое которых также может быть использовано при выполнении команды и в которые помещается результат выполненной операции (приемник результата). В приведенных примерах адресат-источник обозначается буквами *src* или *S* (source - источник), а операнд-приемник *dst* или *D* (destination - приемник). Поле команды, содержащее код операции, обозначается аббревиатурой КОП.

2.2. Формат одноадресных команд

Разряды 15 - 06 содержат код операции, который определяет выполняемую команду. Разряды 05 - 00 образуют 6-разрядное поле, именуемое полем адресации операнда приемника, которое в свою очередь состоит из двух подполей:

1. Разряды 02 - 00 определяют один из восьми РОН, который использует данная команда;
2. Разряды 05 - 03 определяют способ использования выбранного регистра (метод адресации). Причем разряд 03 определяет прямую или косвенную адресацию.



2.3. Формат двухадресных команд

Операции над двумя операндами (такие, как сложение, пересылка, сравнение) выполняются с помощью команд, в которых задаются два адреса. Задание разрядов в полях адресации операндов источника и приемника определяют используемые методы адресации и регистры общего назначения. Формат двухадресной команды имеет следующий вид:



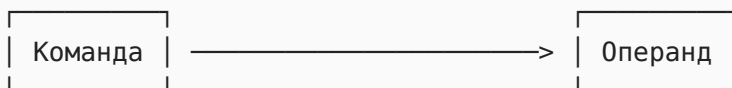
Поле адресации операнда источника используется для выборки операнда источника. Поле адресации операнда приемника используется для выборки операнда приемника и занесения результата. Например, по команде `ADD A, B` содержимое ячейки "A" (операнд источника) складывается с содержимым ячейки "B" (операнд приемника). После выполнения операции сложения в ячейке "B" будет находиться результат операции, а содержимое ячейки "A" не изменится.

2.4. Методы прямой адресации

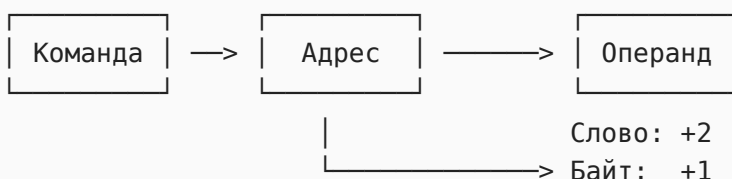
На Рис. 1.7 показаны последовательности операций при выполнении команд с каждым из четырех методов прямой адресации. При регистровом методе адресации операнд находится в выбранном регистре, который может быть использован как накопитель. Так как РОН аппаратно реализованы в ИС центрального процессора, они обладают более высоким быстродействием, чем любая другая память, работающая под управлением процессора. Это их преимуще

ство особенно проявляется при операциях с переменными, к которым необходимо часто обращаться.

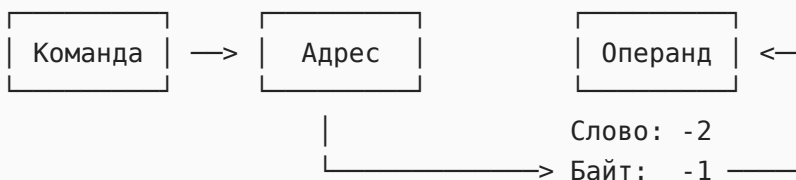
1. Регистровый метод адресации (обозначение R, код $0_8 - 000_2$)



2. Автоинкрементный метод адресации (обозначение (R)+, код $2_8 - 010_2$)



3. Автодекрементный метод адресации (обозначение -(R), код $4_8 - 100_2$)



4. Индексный метод адресации (обозначение X(R), код $6_8 - 110_2$)

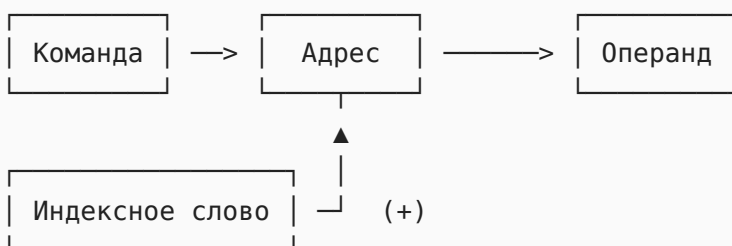


Рис. 1.7 Методы прямой адресации

Регистровый метод адресации

При регистровом методе адресации операнд находится непосредственно в указанном в команде регистре.

Пример 1.

Мнемоника	Восьмеричный код	Название
INC R3	005203	Прибавление единицы

Действие: к содержимому R3 прибавляется единица.

Автоинкрементный метод адресации

При автоинкрементном методе адресации содержимое выбранного регистра является адресом операнда. После выборки операнда содержимое этого регистра автоматически наращивается для обеспечения возможности обращения в дальнейшем к последующей ячейке. При байтовых операциях наращивание происходит на 1, при операциях с полными словами - на 2. Содержимое R6, R7 всегда наращивается на 2.

Автоинкрементный метод адресации особенно удобен при операциях с массивами и стеками. С помощью этого метода можно выбрать элемент таблицы, а затем нарастить указатель для обращения к следующему элементу в таблице.

Пример 2.

Мнемоника	Восьмеричный код	Название
CLR (R5)+	005025	Очистка

Действие: ячейка, адрес которой содержится в R5, очищается, после чего адрес (содержимое R5) увеличивается на 2.

До выполнения операции	После выполнения операции
20000/ 005025	20000/ 005025
30000/ 111116	30000/ 000000
R5/ 030000	R5/ 030002

Автодекрементный метод адресации

Автодекрементный метод адресации также используется для обработки табулированных данных. Однако в отличие от автоинкрементного метода, адресация к ячейкам массива идет в противоположном направлении. При этом методе адресации содержимое выбранного РОН в начале уменьшается (для байтовых команд - на единицу, для команд с полными словами - на два), а затем используется как исполнительный адрес.

Сочетание автоинкрементного и автодекрементного методов адресации может быть эффективно использовано при работе со стеком.

Пример 3.

Мнемоника	Восьмеричный код	Название
INC -(R0)	005240	Прибавление единицы

Действие: содержимое R0 уменьшается на 2 и используется как исполнительный адрес. К операнду, выбранному из ячейки по этому адресу, прибавляется единица.

До выполнения операции	После выполнения операции
100/ 005240	100/ 005240

17774/ 000000	17774/ 000001
R0/ 017776	R0/ 017774

Индексный метод адресации

При индексном методе адресации исполнительный адрес определяется как сумма содержимого выбранного РОН с индексным словом. Этот метод позволяет осуществлять произвольный доступ к элементам структуры данных. Индексное слово содержится в следующей за командным словом ячейке памяти. При индексном методе адресации содержимое выбранного регистра может быть использовано в качестве базы для вычисления серии адресов.

Пример 4.

Мнемоника	Восьмеричный код	Название
CLR 200(R4)	005064	Очистка

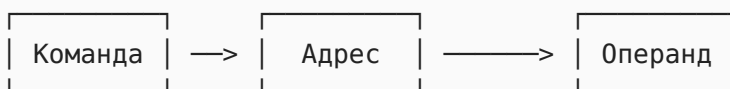
Действие: адрес операнда определяется прибавлением к содержимому R4 кода 200, после чего ячейка с вычисленным адресом очищается.

До выполнения операции	После выполнения операции
1020/ 005064	1020/ 005064
1022/ 000200	1022/ 000200
1200/ 177777	1200/ 000000
R4/ 001000	R4/ 001000

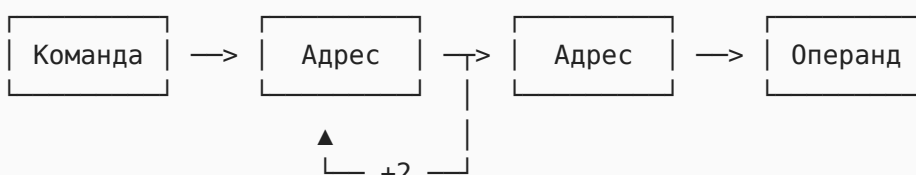
2.5. Методы косвенной адресации

Четыре основных метода могут быть использованы в комбинации с косвенной адресацией. Если при регистровом методе содержимое выбранного регистра является операндом, то при косвенно-регистровом методе это содержимое является адресом операнда. При трёх других косвенных методах вычисленный адрес позволяет выбрать только адрес операнда, а не сам операнд. Эти методы используются при обращении к таблицам, состоящим из адресов, а не из операндов (Рис. 1.8).

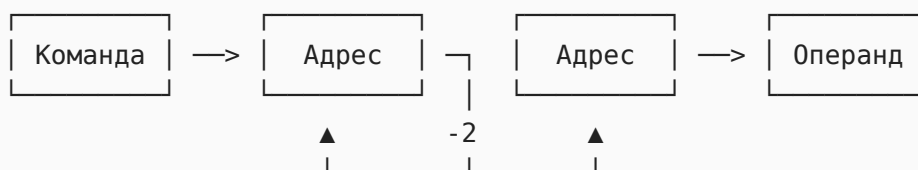
1. Косвенно-регистровый метод адресации (обозначение @R, код 1₈ - 001₂)



2. Косвенно-автоинкрементный метод адресации (обозначение @(R)+, код 3₈ - 011₂)



3. Косвенно-автодекрементный метод адресации (обозначение @-(R), код $5_8 - 101_2$)



4. Косвенно-индексный метод адресации (обозначение @X(R), код $7_8 - 111_2$)

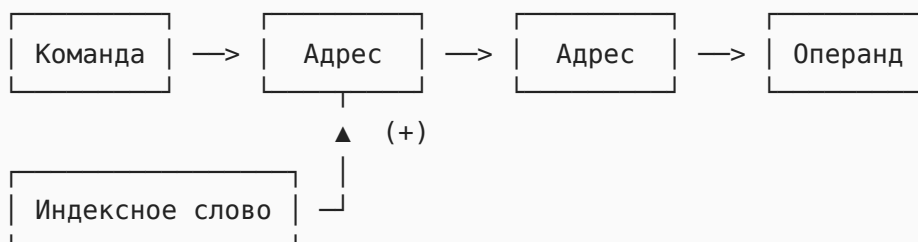


Рис. 1.8 Методы косвенной адресации

2.6. Использование счетчика команд (PC) в качестве РОН

Счетчик команд R7 может быть использован со всеми методами адресации, применяемыми в микро-ЭВМ. Однако наиболее эффективно он используется только с четырьмя из них. Эти методы адресации получили специальные наименования: непосредственный, абсолютный, относительный и косвенно-относительный. Использование этих методов даёт возможность построения программ, работоспособность которых не теряется при перемещении их в любую область памяти.

Восьм. код	Двоичный код	Название	Функция
2	010	Непосредственный	Операнд выбирается из ячейки, следующей за командным словом.
3	011	Абсолютный	Из ячейки, следующей за командным словом, выбирается адрес операнда.
6	110	Относительный	Операнд выбирается из ячейки, адрес которой определяется как сумма содержимого R7 и ячейки, следующей за командным словом.
7	111	Косвенно-относительный	Из ячейки, адрес которой определяется как сумма содержимого R7 и ячейки, следующей за командным словом, выбирается адрес операнда.

Непосредственный метод адресации

Непосредственный метод адресации имеет символическое обозначение #N. Он эквивалентен автоинкрементному методу адресации через счетчик команд R7. Этот метод обеспечивает

экономии времени программиста при составлении программы за счёт возможности помещения константы в ячейку памяти вслед за командным словом.

Пример 6.

Мнемоника	Восьмеричный код	Название
ADD #10, R0	062700	Сложение

Действие: содержимое R0 складывается с числом 10. Результат записывается в R0.

До выполнения операции	После выполнения операции
1020/ 062700	1020/ 062700
1022/ 000010	1022/ 000010
R0/ 000020	R0/ 000030

Примечание. После выборки команды содержимое R7 (адрес этой команды) увеличивается на 2. Так как в поле адреса операнда источника записан код 27, R7 используется как указатель адреса при выборке операнда, после чего содержимое его вновь увеличивается на 2 для указания на следующую команду.

Абсолютный метод адресации

Абсолютный метод адресации имеет символическое обозначение @#A. Он эквивалентен косвенно-автоинкрементной адресации через R7. Этот метод удобен тем, что адрес операнда является его абсолютным адресом (т.е. он остаётся постоянным независимо от места расположения программы в памяти).

Пример 7.

Мнемоника	Восьмеричный код	Название
CLR @#1100	005037	Очистка

Действие: содержимое ячейки, следующей за командой, используется в качестве адреса операнда (в данном случае исполнительным адресом является код 1100). Содержимое ячейки с адресом 1100 очищается.

До выполнения операции	После выполнения операции
20/ 005037	20/ 005037
22/ 001100	22/ 001100
1100/ 177777	1100/ 000000

Относительный метод адресации

Относительный метод адресации имеет символическое обозначение X(PC) или A, где X - исполнительный адрес по отношению к счетчику команд. Этот метод эквивалентен индексной адресации через R7. Индексное слово хранится в следующей за командным словом ячейке

и, будучи сложением с содержимым R7, даёт адрес операнда. Этот метод полезен при написании программы, которая может располагаться в различных местах памяти, так как адрес операнда фиксируется по отношению к содержимому R7. При необходимости перемещения программы в памяти операнд перемещается на то же число ячеек, что и сама команда.

Пример 8.

Мнемоника	Восьмеричный код	Название
INC A	005267	Прибавление единицы

Действие: к операнду, адрес которого определяется сложением содержимого R7 и индексного слова (000054), прибавляется "1".

До выполнения операции	После выполнения операции
1020/ 005267	1020/ 005267
1022/ 000054	1022/ 000054
1024/	1024/
1100/ 000000	1100/ 000001

Косвенно-относительный метод адресации

Косвенно-относительный метод адресации имеет символическое обозначение @X(PC) или @A, где X - адрес ячейки, содержащей исполнительный адрес, по отношению к счетчику команд. Этот метод эквивалентен косвенно-индексной адресации через СК.

3. Обозначения, используемые при описании команд

Описание каждой команды включает: мнемонику, восьмеричный код, формат команды, двоичный код, описание выполнения команды и выработки признаков, специальные пояснения и примеры.

Обозначение	Значение	Обозначение	Значение
R	регистр общего назначения	B	байтовая команда
УС (SP)	указатель стека (R6)	СК (PC)	счетчик команд (R7)
PCП (RS)	регистр состояния процессора	CCП	слово состояния процессора
SRC	источник	SS	поле адресации операнда источника
(SRC)	операнд источника	DD	поле адресации операнда приемника
DST	приемник	NN	смещение (6 двоичных разрядов)

Обозначение	Значение	Обозначение	Значение
(DST)	операнд приемника	XXX	смещение (8 двоичных разрядов)
()	содержимое ячейки	*	«исключающее ИЛИ»
&	логическое умножение («И»)	V	логическое сложение («ИЛИ»)
/=	не равно	=	равно
<>	не равно	Ā	отрицание A («НЕ»)
>=	больше или равно	<=	меньше или равно
Ā	становится равным	PUSH	запись в стек
POP	выборка из стека	*	умножение
**	возведение в степень		

4. Выполнение байтовых команд

Большинство команд ЭВМ оперируют как с полными словами, так и с байтами. Байтовые команды с автоинкрементным или автодекрементным методами адресации для обращения к следующему байту изменяют содержимое указанного регистра на "1". Байтовые команды при регистровом методе адресации производят обработку младшего байта выбранного регистра.

Если старший разряд командного слова (разряд 15) установлен в "1", он указывает, что команда байтовая. Если же в разряде 15 командного слова записан "0", команда оперирует с полным словом.

5. Одноадресные команды

Очистка	CLR	0050DD
	CLRB	1050DD
Действие:	(DST) ← 0	
Описание:	В указанную ячейку записывается нуль. Для байтовой команды нуль записывается в указанный байт.	
Признаки:	N V Z C 0 1 0 0	

Инвертирование	COM	0051DD
	COMB	1051DD
Действие:	$(DST) \leftarrow \sim(DST)$	
Описание:	Содержимое указанной ячейки заменяется его двоичным обратным кодом (каждый разряд, содержащий 0, устанавливается, а каждый разряд, содержащий 1, очищается). Для байтовой команды операция производится по отношению к указанному байту.	
Признаки:	N V Z C * 0 * 1 • N = 1, если <i>результат</i> < 0 • Z = 1, если <i>результат</i> = 0	

Прибавление единицы	INC	0052DD
	INCB	1052DD
Действие:	$(DST) \leftarrow (DST) + 1$	
Описание:	К содержимому указанной ячейки (или байту, если команда байтовая) прибавляется единица.	
Признаки:	N V Z C * * * • N = 1, если <i>результат</i> < 0 • Z = 1, если <i>результат</i> = 0 • V = 1, если <i>операнд</i> = 077777 • C - не изменяется.	

Вычитание единицы	DEC	0053DD
	DECB	1053DD
Действие:	$(DST) \leftarrow (DST) - 1$	
Описание:	Из содержимого указанной ячейки (или указанного байта для байтовых команд) вычитается единица.	
Признаки:	N V Z C * * * • N = 1, если <i>результат</i> < 0 • Z = 1, если <i>результат</i> = 0 • V = 1, если <i>операнд</i> = 100000 • C - не изменяется.	

Изменение знака	NEG	0054DD
	NEGB	1054DD
Действие:	$(DST) \leftarrow -(DST)$	
Описание:	Содержимое указанной ячейки (или байта для байтовых команд) заменяется его двоичным дополнением. Следует заметить, что число 100000 заменяется самим собой, так как не существует соответствующего ему положительного числа.	
Признаки:	N V Z C * * * * • N = 1, если <i>результат</i> < 0 • Z = 1, если <i>результат</i> = 0 • V = 1, если <i>операнд</i> = 100000	

	• $C = 0$, если <i>результат</i> = 0
--	---

Проверка	TST	0057DD
	TSTB	1057DD
Действие:	$(DST) \leftarrow (DST)$	
Описание:	В зависимости от содержимого указанной ячейки (или байта для байтовых команд) устанавливаются или очищаются признаки N и Z.	
Признаки:	N V Z C * 0 * 0 • $N = 1$, если <i>результат</i> < 0 • $Z = 1$, если <i>результат</i> = 0	

Увеличение или уменьшение чисел в кратное степени 2 число раз выполняется с помощью команд арифметического сдвига: ASR - арифметического сдвига вправо и ASL - арифметического сдвига влево. Знаковый разряд операнда (разряд 15) при арифметическом сдвиге вправо восстанавливается. В младший разряд при арифметическом сдвиге влево заносится ноль. Информация, сдвинутая за пределы C-разряда, теряется.

Арифметический сдвиг вправо	ASR	0062DD
	ASRB	1062DD
Действие:	$(DST) \leftarrow$ сдвинутое на одну позицию вправо (DST)	
Описание:	Все разряды операнда сдвигаются вправо на одну позицию. Содержимое знакового разряда восстанавливается. C-разряд загружается содержимым младшего разряда операнда. Таким образом, ASR или ASRB выполняет деление числа со знаком на 2.	
Признаки:	N V Z C	

	* * * * • $N = 1$, если <i>результат</i> < 0 • $Z = 1$, если <i>результат</i> $= 0$ • $V = N * C$ (после сдвига) • C = содержимое младшего разряда указанной ячейки
--	--

Арифметический сдвиг влево	ASL	0063DD
	ASLB	1063DD
Действие:	(DST) ← сдвинутое на одну позицию влево (DST)	
Описание:	Все разряды операнда сдвигаются влево на одну позицию. В младший разряд результата записывается ноль. С-разряд загружается содержимым старшего разряда операнда. Таким образом, ASL или ASLB выполняет умножение числа со знаком на 2.	
Признаки:	N V Z C * * * * • $N = 1$, если <i>результат</i> < 0 • $Z = 1$, если <i>результат</i> $= 0$ • $V = N * C$ (после сдвига) • C = содержимое старшего разряда указанной ячейки	

Для облегчения последовательной проверки и поразрядной обработки операнда используют команды циклического сдвига. Они оперируют со словом операнда и С-разрядом как с со

держимым 17-разрядного регистра с циклическим переносом.

Циклический сдвиг вправо	ROR	0060DD
	RORB	1060DD
Действие:	(DST) ← циклически сдвинутое на одну позицию вправо (DST)	
Описание:	Все разряды операнда циклически сдвигаются на одну позицию вправо. Содержимое младшего разряда загружается в C-разряд, а прежнее содержимое C-разряда загружается в старший разряд результата.	
Признаки:	N V Z C * * * * • N = 1, если <i>результат</i> < 0 • Z = 1, если <i>результат</i> = 0 • V = N*C (после сдвига) • C = содержимое младшего разряда операнда	

Циклический сдвиг влево	ROL	0061DD
	ROLB	1061DD
Действие:	(DST) ← циклически сдвинутое на одну позицию влево (DST)	
Описание:	Все разряды операнда циклически сдвигаются на одну позицию влево. Содержимое старшего разряда загружается в C-разряд, а прежнее содержимое C-разряда загружается в младший разряд результата.	
Признаки:	N V Z C * * * * • N = 1, если <i>результат</i> < 0 • Z = 1, если <i>результат</i> = 0	

- $V = N * C$ (после сдвига)
- C = содержимое старшего разряда операнда

6. Двухадресные команды

Использование двухадресных команд обеспечивает возможность экономии машинного времени и сокращения количества команд в программе. Список двухадресных команд содержит четыре арифметические и четыре логические команды.

Пересылка	MOV	01SSDD
	MOVB	11SSDD
Действие:	$(DST) \leftarrow (SRC)$	
Описание:	Операнд источника (SRC) пересылается по адресу операнда приемника. Прежнее содержимое ячейки DST теряется, содержимое ячейки SRC не изменяется. При операциях с байтами команда MOVB с использованием регистрового метода адресации (единственная среди байтовых команд) расширяет старший разряд младшего байта (расширение знака). Все разряды старшего байта устанавливаются или сбрасываются в зависимости от того, установлен или сброшен старший (знаковый) разряд младшего байта. В других случаях MOVB оперирует с байтами так, как MOV со словами.	
Признаки:	N V Z C * 0 * • $N = 1$, если $(SRC) < 0$ • $Z = 1$, если $(SRC) = 0$ • C - не изменяется	

--	--

Сравнение	CMP	02SSDD
	CMPB	12SSDD
Действие:	(SRC) - (DST)	
Описание:	Сравниваются операнды источника и приемника и, как результат сравнения, изменяются признаки, которые затем могут быть использованы для команд условных переходов. Оба операнда не изменяются. За командой сравнения обычно следует команда условного ветвления. Заметим, что в отличие от команды вычитания при выполнении команды CMP операнды меняются местами, т.е. имеет место (SRC) - (DST), а не (DST) - (SRC).	
Признаки:	N V Z C * * * * • N = 1, если <i>результат</i> < 0 • Z = 1, если <i>результат</i> = 0 • V = 1, если <i>арифметическое переполнение</i> • C = 1, если <i>перенос из старшего разряда</i>	

Сложение	ADD	06SSDD
Действие:	(DST) ← (SRC) + (DST)	
Описание:	Операнд источника (SRC) складывается с операндом приемника (DST) и результат записывается по адресу операнда приемника. Первоначальное содержимое (DST) теряется. Содержимое (SRC) не изменяется. Сложение выполняется в двоичном дополнительном коде. (Байтового аналога нет.)	
Признаки:		

	N V Z C * * * * • N = 1, если <i>результат</i> < 0 • Z = 1, если <i>результат</i> = 0 • V = 1, если <i>арифметическое переполнение</i> • C = 1, если <i>перенос</i>
--	---

Вычитание	SUB	16SSDD
Действие:	(DST) ← (DST) - (SRC)	
Описание:	Из операнда приемника (DST) вычитается операнд источника (SRC) и результат записывается по адресу DST. Первоначальное содержимое DST теряется, а содержимое SRC остаётся без изменения. (Байтового аналога нет.)	
Признаки:	N V Z C * * * * • N = 1, если <i>результат</i> < 0 • Z = 1, если <i>результат</i> = 0 • V = 1, если <i>арифметическое переполнение</i> • C = 1, если <i>перенос из старшего разряда</i>	

Из четырёх логических команд три имеют такой же формат, как и двухадресные арифметические команды. Четвёртая команда имеет специфический формат. Логические команды позволяют осуществлять поразрядную обработку данных.

Проверка разрядов	BIT	03SSDD
	BITB	13SSDD
Действие:	$(DST) \& (SRC)$	
Описание:	Выполняется логическая операция "И" над (SRC) и (DST) с соответствующим изменением признаков. Оба операнда не изменяют своего значения.	
Признаки:	N V Z C * 0 * • N = 1, если <i>результат</i> < 0 • Z = 1, если <i>результат</i> = 0 • C - не изменяется	

Очистка разрядов	BIC	04SSDD
	BICB	14SSDD
Действие:	$(DST) \leftarrow \sim(SRC) \& (DST)$	
Описание:	Каждый разряд операнда (DST), соответствующий установленному разряду операнда (SRC), очищается. Первоначальное содержимое DST теряется, содержимое SRC не изменяется.	
Признаки:	N V Z C * 0 * • N = 1, если <i>результат</i> < 0 • Z = 1, если <i>результат</i> = 0 • C - не изменяется	

--	--

Логическое сложение	BIS	05SSDD
	BISB	15SSDD
Действие:	$(DST) \leftarrow (SRC) \vee (DST)$	
Описание:	Над содержимым SRC и DST выполняется логическая операция "ИЛИ" и записывается результат по адресу DST. Разряды DST устанавливаются в "1", если соответствующие им разряды (SRC) находятся в "1". Прежнее содержимое DST теряется, а содержимое SRC остаётся неизменным.	
Признаки:	N V Z C * 0 * - N = 1, если <i>результат</i> < 0 - Z = 1, если <i>результат</i> = 0 - C - не изменяется	

Исключающее ИЛИ	XOR	74RDD
Действие:	$(DST) \leftarrow R * (DST)$	
Описание:	Над содержимым указанного регистра R и содержимым DST выполняется операция исключающее ИЛИ. Результат записывается в DST. Содержимое регистра R не изменяется.	
Признаки:	N V Z C * 0 * - N = 1, если <i>результат</i> < 0 - Z = 1, если <i>результат</i> = 0 - C - не изменяется	

7. Команды управления программой

К командам управления программой относятся команды ветвления, обращения к подпрограмме, возврата из подпрограммы, безусловного перехода и другие.

7.1. Команды ветвления

Эти команды вызывают ветвление по адресу, являющемуся суммой смещения (умноженного на 2) и текущего содержимого счетчика команд R7, если условие ветвления выполняется.

Смещение показывает, на сколько ячеек нужно перейти относительно текущего содержимого счетчика команд в ту или другую сторону. Так как слова имеют четные адреса, то для получения истинного исполнительного адреса смещение необходимо умножить на два перед прибавлением к счетчику команд R7, который всегда указывает на слово. Старший разряд смещения (разряд 7) является знаковым разрядом. Если он установлен в 1, смещение отрицательное, ветвление происходит в сторону уменьшения адреса (в обратном направлении). Если в разряде 7 содержится 0, смещение - положительное, и ветвление происходит в сторону увеличения адресов (в прямом направлении). Восьмиразрядное смещение позволяет производить ветвление в обратном направлении максимально на 200_8 слов от слова, на которое указывает текущее содержимое СК, и на 177_8 слов в прямом направлении.

Новое содержимое СК = текущее содержимое СК + 2 * XXX (смещение), где текущее содержимое СК = адрес команды ветвления + 2.

Ветвление безусловное	BR	000400 + XXX
<i>Действие:</i>	$(СК) \leftarrow (СК) + 2 * XXX$	
<i>Описание:</i>	С помощью одной команды управление программой передаётся ячейке, адрес которой находится в ограниченной области.	
<i>Признаки:</i>	N V Z C Не изменяются	

Простые условные ветвления

Ветвление, если не равно (нулю)	BNE	001000 + XXX
<i>Действие:</i>	$(СК) \leftarrow (СК) + 2 * XXX$, если $Z = 0$	

<i>Описание:</i>	Проверяется состояние разряда Z и вызывается ветвление, если он очищен. Команда BNE обратна по действию команде BEQ. Вместе с командой BIT она используется для проверки того, что установленные разряды операнда источника соответствуют установленным разрядам операнда приемника. В общем случае она используется для проверки неравенства нулю результата предыдущей операции.
<i>Признаки:</i>	N V Z C Не изменяются

Ветвление, если равно (нулю)	BEQ	001400 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $Z = 1$	
<i>Описание:</i>	Проверяется состояние разряда Z и вызывается ветвление, если он установлен. Команда BEQ обратна по действию команде BNE. Вместе с командой CMP команда BEQ используется для проверки равенства двух величин. Вместе с командой BIT она используется для проверки того, что очищенные разряды операнда источника соответствуют установленным разрядам приемника. В общем случае эта команда используется для проверки равенства нулю результата предыдущей операции.	
<i>Признаки:</i>	N V Z C Не изменяются	

Ветвление, если плюс	BPL	100000 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $N = 0$	
<i>Описание:</i>	Проверяется разряд N и вызывается ветвление, если он очищен. Вместе с командой TSTB она используется для проверки установки разряда 7 (флага готовности) регистра состояния периферийного устройства. В общем случае эта команда используется для проверки положительности результата предыдущей операции. Команда BPL обратна по действию команде BMI.	
<i>Признаки:</i>	N V Z C Не изменяются	

Ветвление, если минус	BMI	100400 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $N = 1$	

<i>Описание:</i>	Проверяется состояние разряда N и вызывается ветвление, если он установлен. Используется для проверки знака (старший разряд) результата предыдущей операции.
<i>Признаки:</i>	N V Z C Не изменяются

Ветвление, если нет арифметического переполнения	BVC	102000 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $V = 0$	
<i>Описание:</i>	Проверяется состояние разряда V и вызывается ветвление, если он очищен.	
<i>Признаки:</i>	N V Z C Не изменяются	

Ветвление, если арифметическое переполнение	BVS	102400 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $V = 1$	
<i>Описание:</i>	Проверяется состояние разряда V и вызывается ветвление, если он установлен. BVS используется для обнаружения арифметического переполнения в результате исполнения предыдущей операции. BVS обратна по действию команде BVC.	
<i>Признаки:</i>	N V Z C Не изменяются	

Ветвление, если нет переноса	BCC	103000 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $C = 0$	
<i>Описание:</i>	Проверяется разряд C и вызывается ветвление, если он очищен.	
<i>Признаки:</i>	N V Z C Не изменяются	

Ветвление, если перенос	BCS	103400 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $C = 1$	

<i>Описание:</i>	Проверяется разряд C и вызывается ветвление, если он установлен. BCS используется для проверки наличия переноса в результате предыдущей операции. Команда BCS обратна по действию команде BCC.
<i>Признаки:</i>	N V Z C Не изменяются

Условные ветвления по результату операций над числами со знаком

Особые комбинации разрядов признаков проверяются с помощью команд условного ветвления по результату операций над числами. Эти команды используются для проверки результатов выполнения команд, в которых операнды рассматриваются как двоичные числа, имеющие знак. Заметим, что отличие в сравнении чисел, имеющих знак, и чисел без знака обусловлено их различным представлением в арифметике, использующей дополнительные коды. Для 16-разрядных чисел, не имеющих знака, последовательность следующая:

Ветвление, если больше или равно (нулю)	BGE	002000 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $N*V = 0$	
<i>Описание:</i>	Вызывается ветвление, если оба разряда признаков N и V установлены или очищены (т.е. если результат операции исключающее ИЛИ над содержимым разрядов N и V равен 0). Таким образом, команда BGE всегда будет вызывать ветвление, если она следует за операцией сложения двух положительных чисел. Команда BGE будет также вызывать ветвление по нулевому результату.	
<i>Признаки:</i>	N V Z C Не изменяются	

Ветвление, если меньше (нуля)	BLT	002400 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $N*V = 1$	
<i>Описание:</i>	Вызывается ветвление, если результат операции исключающее ИЛИ над содержимым разрядов N и V равен 1. Команда BLT обратна по действию команде BGE. Таким образом, команда BLT всегда будет вызывать ветвление, если она следует за операцией сложения двух отрицательных чисел, даже если происходит переполнение. В частности, команда BLT будет всегда вызывать ветвление, если она следует за командой сравнения (CMP) отрицательного операнда и положительного операнда назначения, даже если произошло переполнение. Команда BLT никогда не будет вызывать ветвления, если она следует за командой	

	сравнения (CMP) положительного операнда источника и отрицательного операнда назначения. Она также не будет вызывать ветвления, если результат предыдущей операции равен нулю без переполнения.
<i>Признаки:</i>	N V Z C Не изменяются

Ветвление, если больше (нуля)	BGT	003000 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $Z \vee (N * C) = 0$	
<i>Описание:</i>	Команда BGT подобна команде BGE, за исключением того, что команда BGT не вызывает ветвления по нулевому результату.	
<i>Признаки:</i>	N V Z C Не изменяются	

Ветвление, если меньше или равно (нулю)	BLE	003400 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $Z \vee (N * C) = 1$	
<i>Описание:</i>	Команда BLE подобна команде BLT, однако вызывает ветвление и по нулевому результату.	
<i>Признаки:</i>	N V Z C Не изменяются	

Условные ветвления по результату операций над кодами (числа без знака)

Условные ветвления по результату операции над кодами обеспечивают методы проверки результата операций сравнения операндов, рассматриваемых как величины без знака.

Ветвление, если больше	BHI	101000 + XXX
<i>Действие:</i>	$(CK) \leftarrow (CK) + 2 * XXX$, если $C = 0$ и $Z = 0$	
<i>Описание:</i>	Вызывается ветвление, если предыдущая операция не вызывала переноса или появления нулевого результата. Это происходит при операциях сравнения CMP, когда операнд источника больше операнда приемника.	
<i>Признаки:</i>	N V Z C Не изменяются	

Ветвление, если больше или равно	BHIS	103000 + XXX
Действие:	$(CK) \leftarrow (CK) + 2 * XXX$, если $C = 0$	
Описание:	По своему действию команда BHIS идентична команде BCC. Другая мнемоника вводится только в связи с другим использованием команды.	
Признаки:	N V Z C Не изменяются	

Ветвление, если меньше	BLO	103400 + XXX
Действие:	$(CK) \leftarrow (CK) + 2 * XXX$, если $C = 1$	
Описание:	По своему действию команда BLO идентична команде BCS. Другая мнемоника вводится только в связи с другим использованием команды.	
Признаки:	N V Z C Не изменяются	

Ветвление, если меньше или равно	BLOS	101100 + XXX
Действие:	$(CK) \leftarrow (CK) + 2 * XXX$, если $C = 1$ или $Z = 1$	
Описание:	По своему действию команда BLOS является обратной команде BHI. Ветвление будет происходить, если операнд источника меньше или равен операнду приемника (без знака).	
Признаки:	N V Z C Не изменяются	

Безусловный переход	JMP	0001DD
Действие:	$(CK) \leftarrow (DST)$	
Описание:	Команда JMP обеспечивает возможность перехода на любую команду программы (не ограничиваясь пределами в +177 и -200 слов как команда BR) с использованием всех методов адресации, за исключением регистрового. Использование регистровой адресации вызывает прерывание программы по условию <i>запрещённая команда</i> через адрес вектора 4. Метод косвенной адресации может применяться и вызывает передачу управления программой по адресу, содержащемуся в указанном регистре. Заметим, что команды - это полные слова и поэтому должны выбираться из ячеек с чётными	

	адресами. Команда JMP с косвенно-индексным методом адресации позволяет передать управление по адресу, являющемуся элементом таблицы адресов.
Признаки:	N V Z C Не изменяются

7.2. Команды обращения к подпрограмме и выхода из подпрограммы

Эти команды обеспечивают возможность автоматического вложения подпрограмм, выхода из подпрограммы и многократного входа в подпрограмму. В подпрограммах могут быть обращения к другим подпрограммам (или к самим себе) без специального программного запоминания адресов возврата. Процедура обращения к подпрограмме и выхода из неё не изменяет подпрограмму. Это позволяет использовать одну и ту же подпрограмму несколькими процессорами, осуществляющими прерывание программы.

Обращение к подпрограмме	JSR	004RDD
Действие:	(TMP) ← (DST) — запись содержимого приёмника во внутренний регистр процессора	
	PUSH (SP) ← (R) — запись содержимого указанного регистра в стек	
	(R) ← (PC) — счётчик команд сохраняет адрес следующей за JSR команды в регистр R	
	(PC) ← (TMP) — занесение в счётчик команд начального адреса подпрограммы	
Описание:	При выполнении команды JSR старое содержимое указанного регистра (указатель связи) автоматически засылается в стек, и в регистр поступает новая связующая информация. Таким образом, обращение к подпрограмме, вложенное в подпрограмму на любую глубину, осуществляется с помощью регистра указатель связи. Нет необходимости в том, чтобы задавать максимальную глубину обращения к данной подпрограмме или включать команды запоминания и восстановления указателя связи в каждую подпрограмму. Обращение к подпрограмме по команде JSR может осуществляться с помощью автоинкрементной адресации (если каждый последующий вход в подпрограмму осуществляется через ячейку, адрес которой на 2 больше предыдущего) или индексной адресации (если вход в подпрограмму осуществляется по адресам, расположенным в произвольном порядке). Оба эти метода могут быть также косвенными.	
Признаки:	N V Z C Не изменяются	

Возврат из подпрограммы	RTS	00020R
Действие:	(PC) ← (R)	
	(R) ← (SP) POP	
Описание:	Содержимое регистра R загружается в счётчик команд, после чего извлекается верхний элемент стека и засылается в указанный регистр. Возврат из подпрограммы обычно выполняется через тот же самый регистр, который используется при обращении к ней. Таким образом, вход и выход подпрограммы, обращение к которой осуществляется командой JSR PC, DST, выполняется командой RTS PC, а выход из подпрограммы, обращение к которой осуществлялось командой JSR R5, DST с использованием любого из методов адресации, выполняется командой RTS R5.	
Признаки:	N V Z C Не изменяются	

Вычитание единицы и ветвление	SOB	077RNN
Действие:	(R) ← (R) - 1	
	(PC) ← (PC) - 2 * NN, если результат <> 0	
	(PC) ← (PC), если результат = 0	
Описание:	Содержимое регистра уменьшается на единицу. Если результат не равен нулю, в счётчик команд загружается новое содержимое, определяемое вычитанием удвоенного смещения из текущего содержимого счётчика команд. В команде SOB смещением является шестизначное положительное число. Эта команда может быть эффективно использована для организации различного рода счётчиков, циклов. Следует отметить, что команда SOB не может быть использована для передачи управления в прямом направлении.	
Признаки:	N V Z C Не изменяются	

7.3. Команды прерывания

Команды прерывания обеспечивают возможность обращения к программам управления ввод-выводом, программам отладки и программам, разработанным пользователем. Когда происходит прерывание, текущее содержимое счётчика команд и содержимое регистра состояния процессора записывается в стек. Новое содержимое счётчика команд и регистра состояния процессора загружается из вектора прерывания, состоящего из двух слов. При выходе из прерывания используются команды RTI и RTT, которые восстанавливают СК и РСР, извлекая

их прежнее содержимое из стека. Векторы прерывания расположены по фиксированным, прописанным каждому виду прерывания адресам.

Командное прерывание для системных программ	EMT	104000 - 104377
Действие:	PUSH (SP) ← (RS)	
	PUSH (SP) ← (PC)	
	(PC) ← (30)	
	(RS) ← (32)	
Описание:	Команды EMT имеют коды операций от 104000 до 104377, которые могут быть использованы для передачи информации в моделирующую программу (т.е. информации о функции, которая должна быть выполнена). Вектор прерывания команды EMT находится по адресу 30. Новое содержимое СК берётся из ячейки с адресом 30, а новое содержимое РСР - из ячейки с адресом 32.	
Признаки:	N V Z C Не изменяются	

Возврат из прерывания	RTI	000002
Действие:	(PC) ← (SP) POP	
	(RS) ← (SP) POP	
Описание:	Команда RTI используется для выхода из подпрограмм обслуживания внешних и внутренних прерываний. Содержимое счётчика команд и регистра состояния процессора восстанавливается с помощью стека.	
Признаки:	N V Z C Не изменяются	

7.4. Команды управления процессором

Останов	HALT	000000
Действие:	(PC) ← (SP) POP	
	(RS) ← (SP) POP	
Описание:	Центральный процессор переходит в режим опроса пультового терминала. В счётчике команд сохраняется адрес команды, которая должна быть исполнена следующей.	

Признаки:

N V Z C Не изменяются